

SEKA: Seeking Knowledge Graph Anomalies

Submitted for double-blind review

ABSTRACT

Knowledge Graphs (KGs) form the backbone of many knowledge dependent applications such as search engines, digital personal assistants, and question-answering machines. KGs are constructed either manually or automatically using a variety of extraction techniques applied over multiple data sources. As we cannot guarantee the accuracy of these data sources, and due to the expenses and difficulties involved in data validation prior to KG formation, it is highly likely that there are incorrect and missing triples in a KG, and it is unrealistic to expect a perfectly accurate and complete archive of knowledge. Therefore, a KG will have a trade-off between its coverage and correctness. To address this issue of KG validation, and to identify interesting and unusual triples that are worth investigating, we introduce SEKA, a novel unsupervised approach to detect anomalous entities (nodes) and triples in a KG. SEKA can help improve the correctness of a KG whilst retaining its coverage. We propose an adaption of the Path Rank Algorithm (PRA), named Corroborative Path Rank Algorithm (CPRA), which is a novel and efficient adaptation of PRA that is customized to detect anomalies in KGs. With the use of a one-class Support Vector Machine (SVM), we identify abnormal entities and triples in an unsupervised manner without any user involvement or dependency on external sources. We evaluate our approach using the four real world KGs YAGO-1, KBpedia, Wikidata, and DSKG to demonstrate the ability of our approach in outperforming a baseline approach both in terms of run time and quality of the results. Furthermore, we conduct Knowledge Graph Completion (KGC) over three benchmark KGs FB15K, FB15K-237, and WN18, to demonstrate how anomaly detection can supplement a downstream task such as KGC.

CCS CONCEPTS

• **Computing methodologies** → **Anomaly detection; Unsupervised learning; Support vector machines; Knowledge representation and reasoning; Feature selection.**

KEYWORDS

Unsupervised anomaly detection, binary features, one-class classifier, knowledge graph completion.

ACM Reference Format:

Submitted for double-blind review. 2022. SEKA: Seeking Knowledge Graph Anomalies. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (CIKM '22)*. ACM, New York, NY, USA, 11 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CIKM '22, October 17–22, 2022, Georgia, USA

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-XXXX-X/18/06...\$15.00

<https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

Knowledge Graphs (KGs) are now the backbone of many information systems that require access to structured knowledge [12]. Even though the phrase ‘Knowledge Graph’ has been used in the literature since early 1970s, the modern incarnation occurred since the announcement made by Google in 2012, followed by the adoption of KGs by other industrial giants such as Amazon, eBay, and Facebook [21]. Furthermore, the introduction of linked open data contributed to shifting current research interest towards KGs [12].

Many of the KGs published as linked open data have been created from semi-structured or unstructured data sources [17, 30]. The sizes of KGs such as YAGO [14] and DBpedia [4] make manual curation difficult, thus requiring the use of heuristics for knowledge extraction given a context. However, these heuristics cannot guarantee that a KG is free from anomalies due to the lack of human validation [46]. Linked data which is extracted from semi-structured and unstructured sources is likely to contain anomalies in the form of incorrect RDF (Resource Description Framework) statements and incomplete facts [5, 36]. For example, Wikipedia, which is used as a data source in constructing YAGO and DBpedia, is estimated to contain 2.8% wrong triples [48]. Hence, automatic approaches to detect anomalous triples in a KG are important for the improvement of KG quality [30]. However, it remains a challenging task to detect anomalies on real world KGs, due to their size and their unique data characteristics, including directed triples, relations of different types, and semantic properties [28], thus requiring a more scalable and practical approach for anomaly detection.

Anomaly or outlier detection is the process of identifying unusual patterns or rare occurrences that deviate from the majority in a dataset [3]. In the context of KGs, anomaly detection is important due to its ability to improve the quality of a KG [40] by identifying entities or triples that deviate from the majority due to an associated anomaly. For a given KG, performing anomaly detection as the first step in the cleaning process will provide a much cleansed KG as an input to validation and enrichment methods, thereby improving their performance in terms of quality.

In light of these challenges involving the quality enhancement of KGs, we propose SEKA, an unsupervised anomaly detection approach to identify anomalous entities, and triples in a KG using both structural characteristics of a graph and content of graph’s nodes. In this paper, we will be using the intuitive notion of an anomaly as being interesting or unusual such as semantically incorrect triples, contradicting triples, triples with redundant information, triples with invalid/missing content, and entities with contradicting type information. Also, we use the term *entity* to refer to a node in the KG that represents a real world object (such as a person, movie, dataset, and so on), and we interchangeably use the terms *fact* and *triple* to represent two related entities.

SEKA performs three independent anomaly detection tasks: (1) general fact anomaly detection, which considers the entire KG to detect anomalous triples, (2) target predicate fact anomaly detection, which allows the user to specify a particular predicate (such as

livesIn) among which they wish to identify any anomalous triples, as opposed to considering the entire graph (eliminating the need of traversing the complete graph while constructing customized features based on a target predicate defined by the user), and (3) target type entity anomaly detection, where the user can specify a particular entity type (such as *PERSON*) to detect anomalous entities.

SEKA adopts a variation of the traditional Path Rank Algorithm (PRA), the Corroborative Path Rank Algorithm (CPRA) which we introduce as a novel customization of PRA to achieve the task of anomaly detection. CPRA considers all corroborative (alternative) paths between two entities in a fact (triple) bounded by a specific length to construct binary features. While we consider these paths as features, we mark the existence or non-existence of a path between two entities with a binary *True/False* value, thus generating binary features. To reduce complexity and to improve performance, CPRA adopts a depth-first search bounded by length as opposed to random walks. Using CPRA, we construct a binary feature vector for each triple selected in the KG thus forming a matrix of features. We then prune the feature space for improved performance. We train a one-class Support Vector Machine [38] to perform unsupervised anomaly detection on the feature matrix [8]. Once we have identified anomalies, we can validate them either manually with the involvement of domain experts, or we can remove the identified abnormal triples from the KG and use the refined graph in a downstream task such as Knowledge Graph Completion (KGC).

To evaluate our approach, we conduct several experiments using two sets of KGs. First, we evaluate our approach over the four real world KGs YAGO-1, KBpedia, Wikidata, and DSKG. We selected these KGs to demonstrate the applicability of our approach on KGs from different domains of varying sizes. We introduce INK (INject anomalies to Knowledge graphs) to inject synthetic anomalies to these KGs due to the absence of labeled data (ground truth anomalies). Using these four KGs, we evaluate SEKA without CPRA, but with the traditional PRA [26] which we consider as the baseline. Furthermore, we conduct experiments to evaluate the performance of SEKA in identifying anomalies under the three aforementioned tasks. Second, we evaluate our approach on the three benchmark datasets FB15K, FB15K-237, and WN18, by performing KGC to demonstrate how well our approach helps in improving the quality of KGC results. Third, we also conduct a manual review of the top most abnormal triples identified from YAGO-1, KBpedia, Wikidata, and DSKG.

Contributions: We specifically contribute the following: (1) We introduce SEKA, an unsupervised anomaly detection approach to identify anomalous entities and triples in a KG. Our approach is domain independent, considers both structure and content for anomaly detection, and it is independent from external resources. The ability of our approach to perform three different anomaly detection tasks (given above) allows a user to either find anomalies in the entire KG, within a specific group of predicates, or among a set of entities. (2) We introduce a customized and extended version of the traditional PRA algorithm [26], which is widely used in the task of link prediction. The Corroborative Path Rank Algorithm (CPRA) which we introduced, is customized for anomaly detection, has lower complexity and significantly lower run time compared to

traditional PRA, making CPRA scalable and well suited for anomaly detection in large KGs. (3) Finally, the results of our extensive experimental evaluation demonstrate the suitability of SEKA in achieving the intended task of detecting anomalies in KGs, with an increase of up to 16% in both precision and recall, and a decrease in the run time of up to 23% in comparison with our baseline. Furthermore, our results show how KG refinement via anomaly detection can improve the quality of KGC. Subjecting a cleansed KG for KGC after performing anomaly detection as opposed to performing KGC without any cleaning has an increase in Hits@10 of up to 13% and an increase in MRR of up to 11%.

2 RELATED WORK

We now review related work in the domains of unsupervised anomaly detection, and anomaly detection in graphs and KGs,

To handle the problem of anomaly detection, there have been many techniques developed in the past decade, especially for detecting outliers and anomalies in unstructured data [2, 13, 23]. However, in the real world, data objects are often related to each other and exhibit relationships. In fact, most relational data can be considered as inter-dependent, which requires the consideration of related objects when aiming to identify anomalies [11]. Hence, using graphs for anomaly detection has become increasingly popular [3, 29].

Anomaly detection in graphs can be performed using either supervised, unsupervised, or semi-supervised techniques [20]. However, there is no universally accepted single outlier detection approach, as the choice of a suitable approach depends on the nature of the problem, and the availability of data and their attributes. In the recent past, unsupervised anomaly detection techniques have become prominent among researchers [6, 18, 37], due to the cost involved in both obtaining labelled data, and depending on external resources for supervised learning [13]. Specific techniques have been introduced to detect anomalies in KGs as existing graph-based anomaly detection approaches only consider structural properties, dependent on the graph type, and are not scalable to handle large graphs such as KGs [40].

Anomaly detection in KGs has received much attention as automated methods of constructing KGs prioritize data integrity [19]. There are a few methods for error detection in KGs, where each approach may target specific types of information [35]. There are also approaches taking advantage of entity type information to perform clustering-based outlier detection [9]. However, entity types can either be absent or only partially available in real world KGs. Alternatively, another set of approaches use path ranking [30], and path-based rule mining for error checking [41]. While path ranking methods have limitations in the coverage, path-based rule mining methods have limitations in the quality of the rules.

Most of the recent studies propose to employ supervised classifiers to evaluate every triple based on different features, including entity categories, path features, in/out-degrees, as well as embedding representations of entities and relations [30]. However, ground truth data may not be available to train such classifiers. Alternatively, there have been efforts to utilize external information sources [10, 22, 27, 32, 47], such as related web pages [27] and annotations [24, 28], to facilitate anomaly detection. While having external resources can be valuable for this task, acquiring such

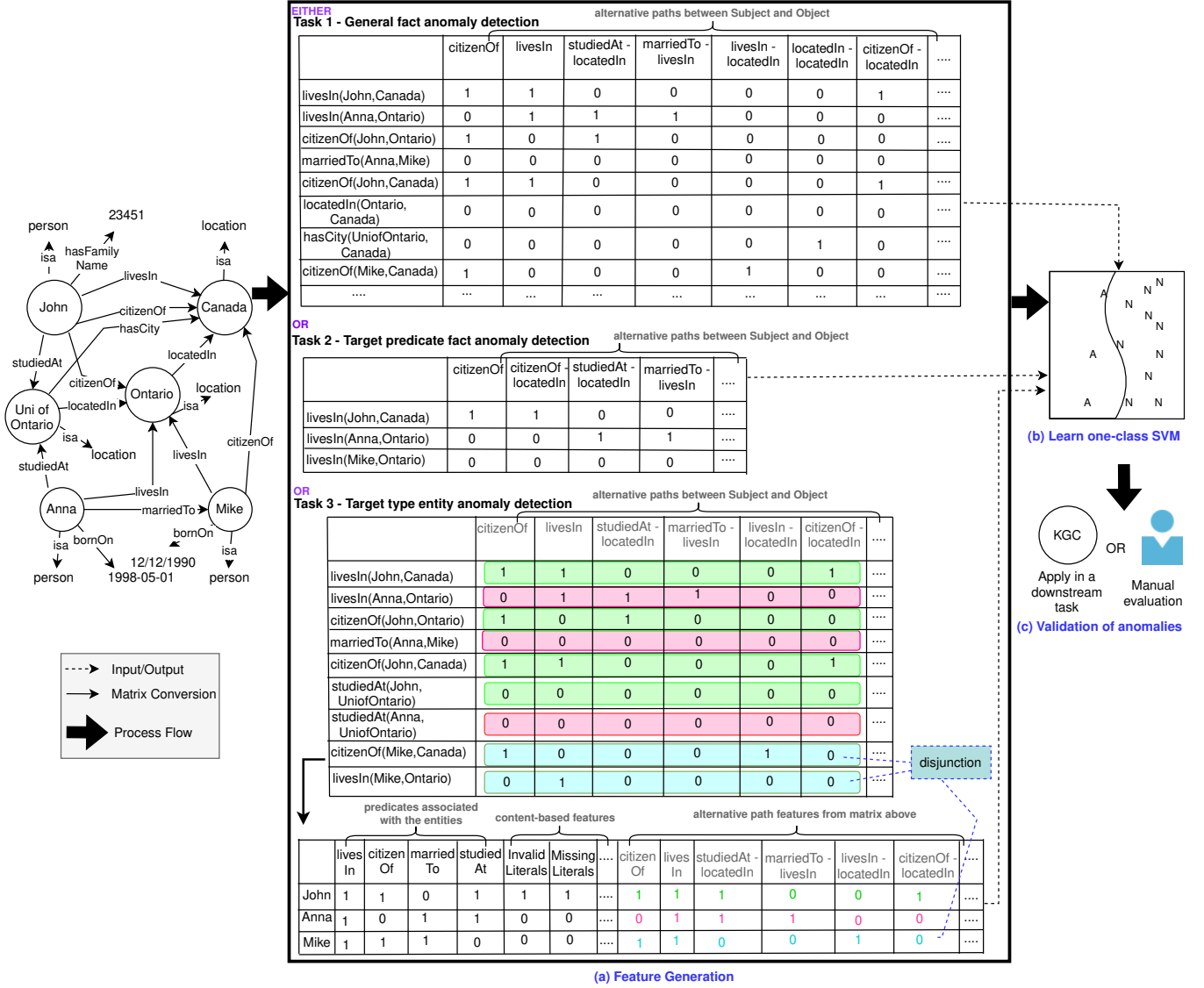


Figure 1: Overview of SEKA, the anomaly detection process to identify anomalous triples and entities in a KG, as described in Section 3. The color usage in the feature matrix in Task 3 represents triples belonging to the same entity.

supplemental information is time consuming and expensive. Furthermore, there are approaches that only aim to identify a single type of anomaly [30], approaches that are KG dependent [34, 49], methods requiring human intervention [24], and embedding methods that can only consider structural properties eliminating the content associated with entities [1, 7, 45, 51].

Hence, none of these approaches can be considered as suitable baselines of our work as none can detect anomalies in both triples and entities while considering both structure and content of the graph in an unsupervised manner.

3 UNSUPERVISED ANOMALY DETECTION

Following recent work in anomaly detection in KGs [25, 40, 43, 52], our aim is to discover abnormal triples and entities that provide

contradicting, semantically incorrect, redundant information, or contain invalid, incomplete, missing information.

Our approach can perform three anomaly detection tasks as shown in Figure 1. (1) General fact anomaly detection considers all triples in the entire graph and aims to identify anomalous triples. (2) Similarly, the task of identifying anomalous target predicate facts aims to identify abnormal triples within a given target predicate. For example, find anomalous triples among the triples with *bornIn* predicate. While this task is similar in execution to the first task of anomaly detection, its goal is to find anomalies related to a specific group of triples without considering the entire graph, thus achieving an advantage over the run time. (3) The third task is to identify anomalous entities given a particular entity type. That is, we identify abnormal entities considering both structural properties

and content associated with the entity node. In this task, we first extract triples related to the specified entity type, where the entity of that type could be either of the to entities in the triples. The goal of this anomaly detection task is to identify abnormal nodes considering their structural properties and associated content.

For example, consider the following three triples: $\langle \text{Marcelina Chissano, bornIn, Mozambique} \rangle$, $\langle \text{Marcelina Chissano, hasPredecessor, Graça Machel} \rangle$, and $\langle \text{Graça Machel, hasSuccessor, Mozambique} \rangle$. These can be seen as a semantic anomaly because the entity *Mozambique* is treated both as a country and as a person. The two triples: $\langle \text{AMG (rapper), produced, Betta Have My Money} \rangle$, $\langle \text{AMG (rapper), produced, Betta Have My Money 2001} \rangle$ are examples of triples providing redundant information, while the triple $\langle \text{Aristotle, bornOnDate, "384-##-##"} \rangle$ contains an invalid date. These are some of the interesting anomalies detected by SEKA in YAGO-1. We will now describe the anomaly detection process of SEKA as outlined in Figure 1.

We consider a directed edge-labelled KG, $G = (V, E)$ containing a set of nodes (or vertices) V and a set of labelled edges E connecting these vertices [40]. The Resource Description Framework (RDF) is a standardised data model based on directed edge-labelled graphs¹. The RDF model defines three types of nodes in a graph: (1) Internationalized Resource Identifiers (IRIs) I which assigns a global identifier for entities I_e and relations I_r on the web (where $I = I_e \cup I_r$); (2) literals L which represents strings and other datatype values; and (3) blank nodes B which are anonymous nodes (not a URI reference or a literal) that do not have an identifier [21]. We therefore have the node set $V = (I_e \cup L \cup B)$, and edge set $E \in V \times I_r \times V$. Each edge $t \in E \in G$ is considered as a RDF triple. A triple (also named as a triplet) contains the three elements subject $s \in S$, predicate $p \in P$, and object $o \in O$. A triple is denoted as (s, p, o) where $(s, o) \in V$, and $(s \times p \times o) \in E$. Furthermore, $s \in (I, B)$, $p \in I_r$, and $o \in (I, L, B)$ [40].

3.1 Feature Generation

In order to perform the aforementioned three anomaly detection tasks, we construct two types of features to identify anomalies in structure and content. The first type (as outlined in Section 3.1.1) characterizes the structural properties of the graph, while the other type (as outlined in Section 3.1.2) characterizes the literals associated with an entity in the graph.

Based on a set of feature generation functions $\mathcal{F}$, where $\mathcal{F}_s$ represents structural feature and $\mathcal{F}_c$ content-based feature, where $\mathcal{F} = \mathcal{F}_s \cup \mathcal{F}_c$, we generate the two feature matrices F_t for fact anomaly detection and F_e for entity anomaly detection, such that $\mathcal{F}_s$ determines the feature matrix F_t , and both $\mathcal{F}_s, \mathcal{F}_c$ determine the feature matrix F_e . While we construct F_t for general fact anomaly detection and target predicate fact anomaly detection, we construct F_e for target type entity anomaly detection.

3.1.1 Generation of structural features. To construct features that highlight structural properties of a KG, we use a variation of the Path Rank Algorithm (PRA) [26]. The traditional PRA can discover complex patterns in relational data by applying logistic regression over paths between nodes that are used as features. Here a path is a sequence of relations $\langle r_1, r_2, \dots, r_l \rangle$ that links two entities [44].

For example, considering the KG in Figure 1, $\langle \text{livesIn, locatedIn} \rangle$ is a path linking *Anna* to *Canada*, through an intermediary node *Ontario*. These paths are extracted through feature selection by performing random walks over the graph. For each triple in the KG, each path is assigned a weight that reflects the probability of arriving at the triple's targeted object, given the triple's subject and the path [15]. PRA is one of the most commonly used approaches for learning inference paths in large KGs [16].

We now propose the Corroborative Path Rank Algorithm (CPRA), a novel extension of the PRA that is customized for the task of anomaly detection. Following are the characteristics of CPRA.

- CPRA considers **all** paths Π between two entities ($s \in S$ and $o \in O$, where $(s, o) \in I_e$) given a particular path length n . To reduce complexity, we bound n as we discuss below. Referring to the KG in Figure 1, between the two entities *John* ($s \in S$) and *Canada* ($o \in O$), there exists three paths, where $\langle \text{livesIn} \rangle$ and $\langle \text{citizenOf} \rangle$ are of $n = 1$ as given in path 3, and $\langle \text{citizenOf, locatedIn} \rangle$ is of $n = 2$ as given in path 4. Here, we also consider the inverse predicate $P_1(s, o) \wedge P_2(o, s)$. We introduce the concept of *half path* (paths of length $n = 0.5$) to determine the occurrence of a given subject $s \in I_e$ and predicate $p \in P$ together with any other object $o \in I_e$ as opposed to the object under consideration. That is, we consider half of a triple, which is the subject and predicate to determine its other occurrences. For example, *John* holds the relationship *citizenOf* with the two entities *Canada* and *Ontario*. We demonstrate this in path 1. We consider these paths as features in F_t . Another half path we consider when constructing the entity features for F_e is the occurrence of a given subject $s \in I_e$ in a particular triple with any other object $o \in I_e$ and predicate $p \in P$ as opposed to the object and predicate under consideration, as demonstrated in path 2. In path 1 and 2, x represents a free variable which is quantified existentially.

$$P_1(s, o) \wedge P_1(s, x), x! = o \quad (1)$$

$$P_1(s, o) \wedge P_2(s, x), x! = o \quad (2)$$

$$P_1(s, o) \wedge P_3(s, o) \quad (3)$$

$$P_1(s, o) \wedge P_4(s, z) \wedge P_5(z, o) \quad (4)$$

- CPRA performs a depth-first search [39] on every path available from the starting node instead of a random walk over the graph to identify $\pi \in \Pi$ between a given $s \in I_e$ and $o \in I_e$. We do not perform random-walk as we are not interested in calculating the random walk probability, which is $P(o|s, \pi \in \Pi)$ the probability of arriving at a particular object $o \in I_e$ given a random walk starting from the subject $s \in I_e$ exactly following all relations in $\pi \in \Pi$ [44]. Since our aim is to identify anomalous triples and entities using the paths that connect these entities, we use *all* $\pi \in \Pi$ between $(s, o) \in I_e$ bounded by n , instead of a probabilistic approach. Furthermore, generating binary features improves the explainability of the anomalies. For example, the triple $\langle \text{Maya, livesIn, Canada} \rangle$ is abnormal due to the path *originatedIn* that exists between *Maya* and *Canada*.

¹<https://www.w3.org/TR/rdf-concepts/#section-Graph-URIfref>

- CPRA helps to perform anomaly detection under the three tasks as per Step (a) in Figure 1. It is applicable either on the entire KG, or on a subset of the KG thus providing an advantage over the run time.

We perform correlation analysis on pairs of features where we remove one feature from a perfectly correlated feature pair to prune the feature space. We also remove features with a single value to reduce the feature space, without depending on any precision-recall measure [26] or the frequency of occurrence [16] for path selection.

As per Figure 1, SEKA can identify anomalies either in the entire graph considering all the triples, or within triples having a specific predicate $p \in P$ (such as *livesIn*, *bornIn*, and so on), or within triples where the subject or entity $(s, o) \in I_e$ is of a particular entity type (such as *PERSON*, *LOCATION*, and so on).

While most of the KGs provide entity type information, we introduced ENTGENE (ENTITY Type GENERator), which operates independently from other external KGs to generate or infer the entity types in their absence. If the entity type information is absent in the KG, we use Named Entity Recognition (NER) in Natural Language Processing (NLP) [31] to generate the type information with the use of Stanford² and Spacy³ libraries in Python. If the type information is partially available in the KG or when the type information cannot be generated (identified as *OTHER* by NER), ENTGENE infers the type using the type information of already identified entities. For example, if the majority of objects $o \in I_e$ in triples with the predicate *livesIn* have the type *LOCATION*, we make this inference for entities with missing type information, where the object of a triple with the predicate *livesIn* should be of type *LOCATION*. While it is possible to have multiple types associated with an entity due to inferencing, any abnormalities related to entity type information will be identified during the process of target type entity anomaly detection.

In F_t , we have one binary feature vector f per triple. While the features are all the alternative (corroborative) paths given a subject $s \in I_e$ and object $o \in I_e$ in a triple, we aggregate all such paths $\pi \in \Pi$ of all the triples in F_t to form the columns of the matrix. If a particular feature exists as a path $\pi \in \Pi$ between a $s \in I_e$ and object $o \in I_e$, the feature value is set to *True* (binary 1), or *False* (binary 0) otherwise.

In F_e , we have one binary feature vector f per entity, where these entities are the subjects $s \in I_e$ of the triples in F_t belonging to a specific entity type. The features of matrix F_e are the aggregation of all predicates $p \in P$ that a particular entity is associated with, all the features from F_t whose binary feature vectors f belong to the triples of the entity (when the entity is a subject $s \in I_e$) where these features are consolidated via disjunction, and the content based features constructed using $\mathcal{F}_e$ as described next.

3.1.2 Generation of content-based features. We generate content-based binary features by determining the presence/absence, validity/invalidity, and so on, of the literals $L \in O$ in triples.

For the feature vector f of F_e , we propose the following binary data quality features.

- Valid literal value: data type of object $o \in L$ matches the data type implied by the predicate $p \in P$ which is set to *True*, if the data type of the object matches the data type implied by the predicate, and *False* otherwise. Our approach learns frequently occurring data type of a particular predicate’s literal and use it for inferencing. For example, using the predicate *hasFamilyName* with the literal *1993-10-10* is incorrect as the object should be of the data type *STRING*, not *DATE*. As another example, consider the anomaly *<Arthur Koestler, bornOnDate, "1905-##-##">* present in YAGO-1.
- Presence of literal value: determines the presence or absence of the object $o \in L$ value in the triple. The feature value is set to *True* if the value is present, and *False* otherwise. For example, the triple *<Azores, hasMotto, "">* in YAGO-1 has an empty object resulting in a *False* feature value.
- Redundant literals: determines the frequency of occurrence of a particular subject $s \in S$ and predicate $p \in P$ together given the object is a literal. The feature is set to *True* if the count of occurrence of a given subject and predicate together is higher than a particular threshold c (mean or median of the counts), and *False* otherwise. For example, the entity *Aggressive Skating* in KBpedia has redundant literals due to the presence of many alternative labels, out of which most have spellings mistakes. Consider the following triples as examples; *<Aggressive Skating, hasLabel, Skating>*, *<Aggressive Skating, hasLabel, Skateing>*, and *<Aggressive Skating, hasLabel, Skat>*
- Subject out-degree: determines the frequency of literals associated with a particular node, where the node is an entity. The feature is set to *True* if the subject out-degree of an entity is higher than a particular threshold c (mean or median of the counts), and *False* otherwise. This feature helps to identify entities rich in information as opposed to entities with a fewer amount of content. For example, in YAGO-1, the entity *Donald Trump* has more information compared to *Marla Maples*.

This step of generating features which is dependent on the CPRA algorithm, uses depth-first search to generate the paths. Hence, the complexity of finding a single path ($n = 1$) is $O(V + E)$, where V corresponds to the nodes and E corresponds to the edges. However, the number of simple paths in a graph can be as large as $O(m!)$ in the entire graph of order m .

As the next step of learning anomalies is independent of which feature matrix F_t or F_e we use, we simply denote F to refer to either of these feature matrices, and similarly we use $\mathcal{F}$ to represent the feature generation functions in general.

3.2 Learn one-class SVM

While binary classification problems require training data with examples from both classes, a variation of the popular support vector machine (SVM) classifier can be used to learn the decision boundary based only on a set of data points and their distribution in an unsupervised manner [38]. The *one-class v-SVM* was originally developed to identify dense areas of high-dimensional distributions. Data points are mapped into a feature space such that a given set

²<https://nlp.stanford.edu/software/CRF-NER.html>

³<https://spacy.io/api/entityrecognizer>

of data points is separated from the origin with a maximum margin [50]. A one-class ν -SVM learns a non-linear decision function where data points that have a non negative distance, $0 \leq d$, from the decision boundary are in the region capturing the majority of data points. This is considered as the *normal* class. Negative distances, $d < 0$, identify data points outside this region and are considered to be the *abnormal* class. A one-class ν -SVM achieves this separation by using a kernel function [40, 42].

In order to identify the most abnormal feature vectors, we train several one-class ν -SVMs using different kernel functions. For each ν -SVM we train, we set $\nu = b/n$, where $n = |F|$. This is a fraction of vectors in the feature matrix F , that a domain expert will be able to investigate, where $1 < b \leq |F|$, and it is named as the *budget*. As we input the feature matrix F to one-class ν -SVM that is generated in step (b), the ν -SVM returns a numerical distance $f.d$ from the decision boundary for each feature vector $f \in F$ [40].

We then classify feature vectors as normal if their distance is $f.d \geq 0$ or abnormal if their distance is $f.d < 0$. Specifically, we generate the set of normal feature vectors, $N = \{f \in F : 0 \leq f.d\}$ and abnormal feature vectors, $A = \{f \in F : 0 > f.d\}$, where $F = N \cup A$ [40].

If we train the ν -SVM with F_t , the the output will be the set of anomalous triples, and with F_e , the the output will be the set of anomalous entities.

The complexity of training a ν -SVM is generally quadratic in the number of training records used, which in our case are the feature vectors in F , and therefore is $O(n^2)$ [38]. Obtaining the class label of a single feature vector depends linearly upon the number of support vectors, where this number can be controlled using the parameter ν [38].

3.3 Validation of anomalies

From the output of the ν -SVM, we can extract the abnormal triples or entities and validate them manually with insights from a domain expert. This manual validation can aid in correcting the identified anomalies to improve KG quality. Furthermore, we can remove these anomalous triples from the KG and subject the refined KG for a downstream task such as Knowledge Graph Completion (KGC) to observe performance improvement.

4 EXPERIMENTAL EVALUATION

In this section, we provide a quantitative and qualitative evaluation of SEKA to demonstrate its state-of-the-art performance. We implemented our approach using Python version 3. All experiments were run on a server with 64-bit Intel Xeon (2.4 GHz) CPUs, 512 GBytes of memory, and Ubuntu 18.04. The program code is available on Dropbox ⁴.

4.1 Datasets

We use two sets of KGs, where the first set includes four real world KGs which we use to evaluate SEKA with the baseline, and also evaluate the performance of SEKA in performing the three anomaly detection tasks as described in Section 3. The second set includes

Table 1: KG summaries.

KG	$ I_e $	$ s \in I_e \times p \times o \in L $	$ s \in I_e \times p \times o \in I_e $
YAGO-1	2,215,094	21,337,521	922,741
KBpedia	62,796	534,032	227,060
Wikidata	14,036,475	53,541,372	51,559,889
DSKG	5,952	22,202	828,086

the benchmark KGs which we use to perform KGC, to demonstrate how KG refinement via anomaly detection can improve KGC.

4.1.1 Experimental KGs. We now provide an overview of the four open KGs YAGO-1⁵, KBpedia⁶, Wikidata⁷ and DSKG⁸, that we used to conduct experiments with our approach as summarized in Table 1.

YAGO contains general knowledge about people, cities, countries, movies, and organizations. This version of YAGO includes the data extracted from the categories and info-boxes of Wikipedia, combined with the taxonomy of WordNet.

KBpedia is an open source KG that combines Wikipedia, Wikidata, schema.org, DBpedia, GeoNames, OpenCyc, and standard UNSPSC products and services into an integrated and computable structure. KBpedia has 98% coverage of Wikidata and nearly complete coverage of Wikipedia.

Wikidata is a free, collaborative, multilingual, secondary database, collecting structured data to provide support for Wikipedia, Wikimedia Commons, other wikis of the Wikimedia movement, and to anyone in the world. It is an open knowledge base that can be read and edited by both humans and machines.

The Data Set Knowledge Graph (DSKG) is a RDF dataset that contains information about datasets and the publications that use those datasets. The metadata of the datasets are modeled in the standard vocabulary data catalog (DCAT) and are based on datasets registered in OpenAIRE and Wikidata.

As these datasets do not have labelled data, we manually inject anomalies. To achieve this, we introduce INK (INject anomalies to Knowledge graphs), which injects the following types of anomalies to a KG.

- Change each of subject, object, and predicate one at a time in a triple while preserving the type. For example, replace predicates used for a person with a predicate that is also used for a person. Ex: $\langle \text{personA}, \text{isMarriedTo}, \text{personB} \rangle$ changed to $\langle \text{personA}, \text{hasChild}, \text{personB} \rangle$.
- Change the entity type of each of subject, object, and predicate one at a time in a triple. For example, replace predicates used for a person with a predicate that is not used for a person. Ex: $\langle \text{personA}, \text{isMarriedTo}, \text{personB} \rangle$ changed to $\langle \text{personA}, \text{livesIn}, \text{personB} \rangle$.

⁵<https://yago-knowledge.org/downloads/yago-1>

⁶<https://kbpedia.org/>

⁷https://www.wikidata.org/wiki/Wikidata:Main_Page

⁸<http://dskg.org/>

⁴https://www.dropbox.com/sh/0k59vbc81ya71z/AAAxWoNRAyXDA86-TvhVQU_Aa?dl=0

4.1.2 Benchmark KGs. In this section, we provide an overview of the three benchmark datasets FB15K⁹, FB15K-237¹⁰, and WN18¹¹, that we used to perform the comparative evaluation.

The FB15K dataset is introduced in translating embeddings for modeling multi-relational data. It has triples and textual mentions of Freebase entity pairs. It contains a total of 592,213 triplets, and 14,951 entities with 1,345 different relations. When creating the dataset, a reverse edge with reversed relation types are created for each edge by default. FB15K-237 is a subset of FB15K where inverse relations are removed. This contains 310,079 triples, and 14,541 entities with 237 different relations. The WN18 dataset is also introduced in translating embeddings for modeling multi-relational data. It is an English database, which can be considered as a dictionary, as well as a thesaurus. *Synsets* is a term which reflects sets of nouns, verbs, adjectives, and so on, which are actually the database's entities. Relations define lexical connections between these entities. It includes the full 18 relations scraped from WordNet for approximately 41,000 synsets, resulting in 141,442 triplets.

4.2 Parameter Settings

We now discuss the parameter settings used in SEKA. To eliminate the need of having user involvement during the anomaly detection process, our approach is designed in such a way that it has a minimal number of parameters that need to be manually set.

In the step of constructing features as explained in Section 3.1.1, to determine the path length to traverse in depth-first search, we use n where it is by default set as $n = 0.5, 1, 2$. Hence, no user involvement is required to determine the value of n .

In the step of identifying anomalies, the ν -SVM requires configurations for the two parameters, its kernel and ν . In order to obtain robust results, we train four ν -SVMs with the four kernels RBF, poly, linear, and sigmoid. This ensures our classification outcomes are not biased due to the use of a single kernel function. Hence, the user does not have to configure the kernel for training. The ratio of normal to abnormal classified feature vectors can be specified using the ν parameter introduced in Section 3.2, which can be set by a user according to the budget b of how many abnormal feature vectors she/he can investigate further.

4.3 Results and Discussion

We conducted six different experiments to demonstrate the performance of SEKA. (1) We first conducted experiments with the baseline of our approach, the traditional PRA. We ran both PRA and CPRA on the experimental KGs with synthetic anomalies, and compare the run time, precision, and recall. (2) Next, we injected different percentages of anomalies to the four experimental KGs, to assess the effect of different anomaly proportions on the performance of SEKA in terms of run time, precision, and recall. (3) As the third experiment, we compare the run time, precision, and recall values obtained by SEKA during general fact anomaly detection, and target predicate fact anomaly detection, with synthetic anomalies. (4) Our fourth experiment is to assess the performance of target type entity anomaly detection with synthetic anomalies,

in terms of run time, precision, and recall. (5) Next, we compare the proposed approach SEKA with widely adopted basic graph embedding models (RotatE, TransE, and Simple) using these datasets, and demonstrate how anomaly detection improves the quality of a downstream task such as Knowledge Graph Completion (KGC). (6) Finally, we perform manual evaluation on the four experimental KGs without any anomaly injections, where we manually verify the anomalous nature of some of the abnormal triples.

4.3.1 Baseline evaluation. We consider traditional PRA as the baseline of our work. We conducted experiments under general fact anomaly detection on the four experimental KGs with 10% anomalies injected in to each KG. We ran two rounds of experiments for each KG, where we first assessed the performance of anomaly detection with PRA for feature generation, and next we used CPRA for feature generation. Here we assessed the run time and the quality of the anomalies of probabilistic feature generation Vs. binary feature generation. The experimental results are available on Table 2.

Table 2: Comparison of performance between PRA and CPRA for general fact anomaly detection with 10% anomalies injected.

KG	Approach	Run time (min)	Precision	Recall
YAGO-1	PRA	178	0.85	0.84
	CPRA	121	0.99	0.99
KBpedia	PRA	88	0.80	0.79
	CPRA	48	0.82	0.81
Wikidata	PRA	301	0.79	0.78
	CPRA	232	0.81	0.80
DSKG	PRA	137	0.92	0.90
	CPRA	72	0.99	0.99

As can be seen from Table 2, CPRA outperforms PRA for all four KGs with less run time, and higher precision, recall values. This demonstrates the suitability of CPRA in generating the features for anomaly detection, over PRA which is dedicated for link prediction task. Under both approaches, Wikidata has the largest run time as it is the KG with the largest number of entities, and triples of the form $(s, o) \in I_e$. Similarly, KBpedia has the lowest run time as it has the smallest number of triples of the form $(s, o) \in I_e$.

4.3.2 Evaluation of general fact anomaly detection. We used the two measures precision and recall to evaluate SEKA in performing general fact anomaly detection on the four experimental datasets. We conducted the evaluation under three different percentages of anomalies injected in to the KGs.

As per the results in Table 3, our approach performs well in identifying anomalies in all four KGs under all three anomaly percentages. SEKA performs the best in DSKG with 10% and 20% anomalies, while it performs the best in YAGO-1 with 30% anomalies.

4.3.3 Evaluation of target predicate fact anomaly detection. The purpose of this experiment is to demonstrate the advantage of having an approach that can consider a subset of the KG as opposed to considering the entire graph for anomaly detection, because in

⁹<https://docs.dgl.ai/en/latest/generated/dgl.data.FB15kDataset.html>

¹⁰<https://docs.dgl.ai/en/latest/generated/dgl.data.FB15k237Dataset.html>

¹¹<https://docs.dgl.ai/en/latest/generated/dgl.data.WN18Dataset.html>

Table 3: Results obtained by SEKA for general fact anomaly detection with 10%, 20% and 30% anomalies injected to each KG.

KG	10% anomalies			20% anomalies			30% anomalies		
	anomaly count	precision	recall	anomaly count	precision	recall	anomaly count	precision	recall
YAGO-1	92,274	0.99	0.99	184,548	0.90	0.89	276,822	0.86	0.87
KBpedia	22,706	0.82	0.81	45,412	0.70	0.70	68,118	0.72	0.72
Wikidata	5,155,989	0.81	0.80	10,311,978	0.79	0.78	15,467,967	0.72	0.71
DSKG	82,809	0.99	0.99	165,617	0.95	0.94	248,425	0.82	0.81

the real world applications, subjecting the entire graph multiple times for anomaly detection can be costly.

In Table 4, we compare the precision, recall and run time obtained for general fact anomaly detection (named as task 1) with the average precision, recall and run time obtained for target predicate fact anomaly detection (named as task 2). For task 1, we corrupted 10% of the triples. Similarly, for task 2, we extracted the ten most infrequent predicates, and corrupted 10% of the facts belonging to each predicate. Our aim here is to show the capability of our approach in detecting anomalies of a set of target predicates given their rare occurrence.

As can be seen from the results in Table 4, considering a target predicate for anomaly detection is much faster than considering the entire graph for anomaly detection. This is evident from the low average run times (in task 2) obtained for each KG compared to the run times of task 1. Furthermore, high precision and recall values demonstrate the capability of our approach in identifying anomalies of rarely occurring predicates. Hence, this implies that our approach is well suited for target predicate fact anomaly detection irrespective of the frequency of occurrence of a predicate.

Table 4: Results obtained by SEKA for general fact anomaly detection (Task 1) Vs. target predicate fact anomaly detection (Task 2).

KG	Measure	Task 1	Task 2 (Avg. values)
YAGO-1	Precision	0.99	0.99
	Recall	0.99	0.99
	Run time (min)	121	58
KBpedia	Precision	0.82	0.83
	Recall	0.81	0.82
	Run time (min)	48	15
Wikidata	Precision	0.81	0.84
	Recall	0.80	0.81
	Run time (min)	232	67
DSKG	Precision	0.99	0.99
	Recall	0.99	0.99
	Run time	72	36

4.3.4 Evaluation of target type entity anomaly detection. Under this experiment, we observe the performance of our approach in detecting anomalous entities. An anomalous entity is one that has anomalous triples of the form $(s, o) \in I_e$, and triples of the form $s \in I_e$ and $o \in L$.

Table 5: Results obtained by SEKA for target type entity anomaly detection.

KG	Avg. run time (min)	Avg. precision	Avg. recall
YAGO-1	285	0.93	0.92
KBpedia	127	0.92	0.90
Wikidata	308	0.90	0.90
DSKG	80	0.91	0.90

For this experiment, we considered each entity type in the KG, where we randomly selected hundred entities from each type for corruption. We corrupted 50% of the triples belonging to each selected entity, and ran anomaly detection on each entity type of each KG. Then, averaged the results to obtain Table 5. We corrupted the literals associated with the entities by corrupting their data types to introduce invalid values, and removing values to make them missing.

As per Table 5, our approach has very high results obtained for precision and recall demonstrating its ability to identify anomalous entities. The run time of this approach is relatively high compared to other two anomaly detection approaches, because here we consider literals associated with the entities. For example, Wikidata has the highest number of literals thus taking the longest to execute.

4.3.5 Evaluation on benchmark KGs. We performed KGC on the three benchmark KGs discussed in section 4.1.2 to demonstrate how performing anomaly detection prior to KGC can improve the performance of KGC. We present the results obtained for three of the most widely used embedding models TransE, RotatE, and SimpleE in Table 6. To evaluate performance, we use Hits@10 and MRR metrics.

As per Table 6, we provide results under three particulars. We performed KGC on the original KGs without any changes, which is given as *Original*. Next, we randomly removed some of the triples from the KGs and performed KGC. These results are provided as *Random*. Finally, we performed anomaly detection on the KGs, removed the abnormal triples, and then conducted KGC. These results are available as *AD*.

From the results reported in Table 6, we can note that the best results for all three KGs are from the scenario of conducting anomaly detection. Hence, it is evident that performing anomaly detection on a KG can improve the quality of KGC.

4.3.6 Review of Anomalies. Herewith, we provide a discussion of some of the interesting anomalies in YAGO-1, KBpedia, Wikidata, and DSKG that we identified during manual evaluation.

Table 6: Results obtained by SEKA for KGC on the three benchmark KGs.

Model	Measure	Particular	FB15K	FB15K-237	WN18
TransE	Hits@10	Original	0.75	0.47	0.94
		Random	0.74	0.46	0.92
		AD	0.76	0.56	0.97
	MRR	Original	0.50	0.29	0.86
		Random	0.42	0.29	0.86
		AD	0.60	0.40	0.88
RotatE	Hits@10	Original	0.88	0.53	0.96
		Random	0.62	0.52	0.95
		AD	0.91	0.66	0.97
	MRR	Original	0.80	0.34	0.95
		Random	0.79	0.32	0.94
		AD	0.81	0.35	0.97
SimpleE	Hits@10	Original	0.92	0.88	0.95
		Random	0.91	0.87	0.95
		AD	0.93	0.90	0.97
	MRR	Original	0.90	0.78	0.95
		Random	0.88	0.77	0.94
		AD	0.92	0.80	0.97

Consider the triple *<Marcelina Chissano, bornIn, Mozambique>* which became abnormal due to the two alternative paths *<hasPredecessor, hasPredecessor>* and *<hasPredecessor, hasSuccessor>*. Investigating this anomaly further highlights the triples *<Marcelina Chissano, hasPredecessor, Graça Machel>*, *<Graça Machel, hasPredecessor, Mozambique>*, and *<Graça Machel, hasSuccessor, Mozambique>*. While the predicate *bornIn* is associated with a *LOCATION* as the object, the two predicates *hasPredecessor* and *hasSuccessor* are also associated with the same object which has led to this abnormality as the rest of the triples labeled normal holds the relationships *hasPredecessor* and *hasSuccessor* with an object of type *PERSON*.

The entities *Romeo (rapper)*, *Roy Jones, Jr.*, and *Josh Gracin* have the predicates *bornIn* and *originatesFrom* with the same object. This conflicting usage of the predicate is because these entities are musicians, and have issued albums under their names, thus considering the composer and album as one entity.

As another example, consider the triples *<Marla Maples, hasChild, Donald Trump>* and *<Marla Maples, isMarriedTo, Donald Trump>* which are not abnormal independently, but contradicting when considered as a pair.

The majority of KBpedia’s anomalies are content-based as opposed to structural anomalies. For example, the triples *<Null SQL, hasLabel, "">*, *<-n-a, hasDefinition, "">*, and *<-n-a, hasLabel, "">* have an empty object. Furthermore, the following triples *<12 - Dimethoxybenzene, hasLabel, "Veratrol">* and *<12 - Dimethoxybenzene, hasLabel, "Veratrole">* provide redundant information due to minor variations in the spellings.

In DSKG, the triple *<person/11203, hasName, A.>* is anomalous due to the abnormality in the object. While the triple should represent the name of the person identified by the number *11203*, the name provided is a single letter which causes the anomaly. While

most of the datasets have a single website to provide more information about the dataset known as the landing page, the entity *dataset/1* in DSKG has three landing pages. Even though the three triples related to this dataset have no anomaly in nature, they were identified as abnormal triples since this is the only dataset with three landing pages out of which one is a broken link. Similarly, the dataset *dataset/411* has the highest number of creators, which is thirteen. None of the other datasets have this many creators thus making the triples related to *dataset/411* abnormal.

Similar to KBpedia, Wikidata does not have many structural anomalies. Most of the anomalies are associated with the literals. The structural anomalies are related to entities rich in information (triples) as opposed to the majority with a limited amount of information. For example, the entity *L10551* has five triples associated with it, where as other entities have at most two triples. While the triples itself are not abnormal, the scenario of one entity having many facts compared to the rest is anomalous. In wikidata, literal based anomalies are mostly due to absence of values, or presence of a numerical value when a string is required. For example, the entity *L158675-F1* has an empty value for its label.

5 CONCLUSION AND FUTURE WORK

In this paper, we have present an approach to study the problem of discovering anomalous triples and entities in a Knowledge Graph (KG) in an unsupervised manner. Our approach can identify anomalies related to both structure and content of the KG, it is KG independent, and has the ability to identify a multitude of anomalies without depending on any external sources. A triple is considered anomalous if it is semantically incorrect, contradicting, has incorrect entity type information, provides redundant information, or has an invalid/missing literal.

We conducted six different experiments using four real world KGs with synthetic anomalies, and three benchmark KGs to demonstrate the state-of-the-art performance of our approach in anomaly detection. The results of the experimental evaluation show an increase of up to 16% in both precision and recall, and a decrease in the run time of up to 23% in comparison with the baseline. Furthermore, the results show how KG refinement via anomaly detection can improve the quality of KGC. Subjecting a cleansed KG for KGC after performing anomaly detection as opposed to performing KGC without any cleaning has an increase in Hits@10 of up to 13% and an increase in MRR of up to 11%.

As for future work, we aim to generalize the identified anomalies with the use of a constraint-based language such as SHACL [33]. Once we generalize anomalies, we can use them as validations during the process of KG construction to filter triples. Also we aim to conduct more baseline experiments with existing graph anomaly detection techniques.

REFERENCES

- [1] Farhad Abedini, Mohammad Reza Keyvanpour, and Mohammad Bagher Menhaj. 2020. Correction Tower: A general embedding method of the error recognition for the knowledge graph correction. *International Journal of Pattern Recognition and Artificial Intelligence* 34, 10 (2020), 2059034.
- [2] Shikha Agrawal and Jitendra Agrawal. 2015. Survey on anomaly detection using data mining techniques. *Procedia Computer Science* 60 (2015), 708–713.
- [3] Leman Akoglu, Hanghang Tong, and Danai Koutra. 2015. Graph based anomaly detection and description: a survey. *Data mining and knowledge discovery* 29, 3 (2015), 626–688.
- [4] Sören Auer, Christian Bizer, Georgi Kobilarov, Jens Lehmann, Richard Cyganiak, and Zachary Ives. 2007. Dbpedia: A nucleus for a web of open data. In *The semantic web*. Springer, Berlin, Heidelberg, 722–735.
- [5] Iti Bansal, Sudhanshu Tiwari, and Carlos R. Rivero. 2020. The Impact of Negative Triple Generation Strategies and Anomalies on Knowledge Graph Completion. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (Virtual Event, Ireland) (CIKM '20)*. ACM, New York, USA, 45–54. <https://doi.org/10.1145/3340531.3412023>
- [6] Paul Bergmann, Michael Fauser, David Sattlegger, and Carsten Steger. 2019. MVTec AD—A comprehensive real-world dataset for unsupervised anomaly detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. IEEE, New York, 9592–9600.
- [7] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Duran, Jason Weston, and Oksana Yakhnenko. 2013. Translating embeddings for modeling multi-relational data. *Advances in neural information processing systems* 26 (2013), 9.
- [8] Peter Christen, Ross W. Gayler, Khoi-Nguyen Tran, Jeffrey Fisher, and Dinusha Vatsalan. 2016. Automatic Discovery of Abnormal Values in Large Textual Databases. *Journal of Data and Information Quality* 7, 1–2 (2016), 1–35.
- [9] Jeremy Debattista, Christoph Lange, and Sören Auer. 2016. A preliminary investigation towards improving linked data quality using distance-based outlier detection. In *Joint international semantic technology conference*. Springer, Cham, 116–124.
- [10] Xin Dong, Evgeniy Gabrilovich, Jeremy Heitz, Wilko Horn, Ni Lao, Kevin Murphy, Thomas Strohmann, Shaohua Sun, and Wei Zhang. 2014. Knowledge vault: A web-scale approach to probabilistic knowledge fusion. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*. ACM, New York, USA, 601–610.
- [11] William Eberle and Lawrence Holder. 2015. Scalable anomaly detection in graphs. *Intelligent Data Analysis* 19, 1 (2015), 57–74.
- [12] Lisa Ehrlinger and Wolfram Wöß. 2016. Towards a Definition of Knowledge Graphs. *SEMANTiCS (Posters, Demos, SuCCESS)* 48, 1–4 (2016), 2.
- [13] Eleazar Eskin, Andrew Arnold, Michael Prerau, Leonid Portnoy, and Sal Stolfo. 2002. A geometric framework for unsupervised anomaly detection. In *Applications of data mining in computer security*. Springer, Boston, MA, 77–101.
- [14] M Fabian, Kasneci Gjergji, WEIKUM Gerhard, et al. 2007. Yago: A core of semantic knowledge unifying wordnet and wikipedia. In *16th International world wide web conference, WWW*. ACM, New York, USA, 697–706.
- [15] Romanos Fasoulis, Konstantinos Bougiatiotis, Fotis Aisopos, Anastasios Nentidis, and Georgios Paliouras. 2020. Error detection in Knowledge Graphs: Path Ranking, Embeddings or both. *arXiv preprint arXiv:2002.08762* (2020).
- [16] Matt Gardner, Partha Talukdar, Jayant Krishnamurthy, and Tom Mitchell. 2014. Incorporating vector space similarity in random walk inference over knowledge bases. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*. Association for Computational Linguistics, USA, 397–406.
- [17] Garima Gaur, Arnab Bhattacharya, and Srikanta Bedathur. 2020. How and Why is An Answer (Still) Correct? Maintaining Provenance in Dynamic Knowledge Graphs. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (Virtual Event, Ireland) (CIKM '20)*. ACM, New York, USA, 405–414. <https://doi.org/10.1145/3340531.3411958>
- [18] Dong Gong, Lingqiao Liu, Vuong Le, Budhaditya Saha, Moussa Reda Mansour, Svetha Venkatesh, and Anton van den Hengel. 2019. Memorizing normality to detect anomaly: Memory-augmented deep autoencoder for unsupervised anomaly detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. IEEE, 1705–1714.
- [19] Stefan Heindorf, Martin Potthast, Benno Stein, and Gregor Engels. 2016. Vandalism detection in wikidata. In *Proceedings of the 25th ACM International Conference on Information and Knowledge Management*. ACM, New York, USA, 327–336.
- [20] Victoria Hodge and Jim Austin. 2004. A Survey of Outlier Detection Methodologies. *Artificial Intelligence Review* 22, 2 (2004), 85–126.
- [21] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard De Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, et al. 2021. Knowledge graphs. *ACM Computing Surveys (CSUR)* 54, 4 (2021), 1–37.
- [22] Elwin Huaman, Amar Tauqueer, and Anna Fensel. 2021. Towards Knowledge Graphs Validation Through Weighted Knowledge Sources. In *Iberoamerican Knowledge Graphs and Semantic Web Conference*. Springer, Cham, 47–60.
- [23] Shubham Jain, Amy de Buitléir, and Enda Fallon. 2020. Unsupervised noise detection in unstructured data for automatic parsing. In *2020 16th International Conference on Network and Service Management (CNSM)*. IEEE, 1–5.
- [24] Manuela Noyantara Jeyaraj, Srinath Perera, Malith Jayasinghe, and Nadheesh Jihan. 2019. Probabilistic error detection model for knowledge graph refinement. In *Proceedings of the International Conference on Computational Linguistics and Intelligent Text Processing (CiCLing'19)*. Springer, Switzerland.
- [25] Bin Jia, Cailing Dong, Z. Chen, Kuo-Chu Chang, Nichole Sullivan, and Genshe Chen. 2018. Pattern Discovery and Anomaly Detection via Knowledge Graph. In *2018 21st International Conference on Information Fusion (FUSION)*. IEEE, 2392–2399. <https://doi.org/10.23919/ICIF.2018.8455737>
- [26] Ni Lao, Tom Mitchell, and William Cohen. 2011. Random walk inference and learning in a large scale knowledge base. In *Proceedings of the 2011 conference on empirical methods in natural language processing*. Association for Computational Linguistics, Stroudsburg, USA, 529–539.
- [27] Jens Lehmann, Daniel Gerber, Mohamed Morsey, and Axel-Cyrille Ngonga Ngomo. 2012. Defacto-deep fact validation. In *International semantic web conference*. Springer, Berlin, Heidelberg, 312–327.
- [28] Yezi Liu. 2021. *Error Detection in Knowledge Graphs*. Ph.D. Dissertation. Graduate and Professional School of Texas A and M University.
- [29] Xiaoxiao Ma, Jia Wu, Shan Xue, Jian Yang, Chuan Zhou, Quan Z Sheng, Hui Xiong, and Leman Akoglu. 2021. A comprehensive survey on graph anomaly detection with deep learning. *IEEE Transactions on Knowledge and Data Engineering* (2021).
- [30] André Melo and Heiko Paulheim. 2017. Detection of relation assertion errors in knowledge graphs. In *Proceedings of the Knowledge Capture Conference*. ACM, New York, USA, 1–8.
- [31] David Nadeau and Satoshi Sekine. 2007. A survey of named entity recognition and classification. *Linguistic Investigations* 30, 1 (2007), 3–26.
- [32] Satoshi Nakamura, Shinji Konishi, Adam Jatowt, Hiroaki Ohshima, Hiroaki Kondo, Taro Tezuka, Satoshi Oyama, and Katsumi Tanaka. 2007. Trustworthiness analysis of web search results. In *International Conference on Theory and Practice of Digital Libraries*. Springer, Berlin, Heidelberg, 38–49.
- [33] Pouya Ghiasnezhad Omeran, Kerry Taylor, Sergio Rodriguez Mendez, and Armin Haller. 2021. Learning SHACL Shapes from Knowledge Graphs. *Semantic Web* (2021).
- [34] Heiko Paulheim. 2017. Data-driven joint debugging of the dbpedia mappings and ontology. In *European Semantic Web Conference*. Springer, Cham, 404–418.
- [35] Heiko Paulheim. 2017. Knowledge graph refinement: A survey of approaches and evaluation methods. *Semantic web* 8, 3 (2017), 489–508.
- [36] Heiko Paulheim and Christian Bizer. 2014. Improving the quality of linked data using statistical distributions. *International Journal on Semantic Web and Information Systems (IJSWIS)* 10, 2 (2014), 63–86.
- [37] Thomas Schlegl, Philipp Seeböck, Sebastian M Waldstein, Georg Langs, and Ursula Schmidt-Erfurth. 2019. f-AnoGAN: Fast unsupervised anomaly detection with generative adversarial networks. *Medical image analysis* 54 (2019), 30–44.
- [38] Bernhard Schölkopf, John C Platt, John Shawe-Taylor, Alex J Smola, and Robert C Williamson. 2001. Estimating the support of a high-dimensional distribution. *Neural Computation* 13, 7 (2001), 1443–1471.
- [39] Robert Sedgewick. 2001. *Algorithms in C, part 5: graph algorithms*. Pearson Education, USA.
- [40] Asara Senaratne, Pouya Ghiasnezhad Omeran, Graham Williams, and Peter Christen. 2021. Unsupervised Anomaly Detection in Knowledge Graphs. In *The 10th International Joint Conference on Knowledge Graphs (Virtual Event, Thailand) (IJCKG'21)*. ACM, New York, USA, 161–165. <https://doi.org/10.1145/3502223.3502246>
- [41] Baoxun Shi and Tim Weninger. 2016. Discriminative predicate path mining for fact checking in knowledge graphs. *Knowledge-based systems* 104 (2016), 123–133.
- [42] Vladimir Vapnik. 2000. *The nature of statistical learning theory*. Springer, New York, USA.
- [43] Nathan Vaska and Victoria Helus. 2022. Context-Dependent Anomaly Detection with Knowledge Graph Embedding Models. *arXiv preprint arXiv:2203.09354* (2022).
- [44] Quan Wang, Jing Liu, Yuanfei Luo, Bin Wang, and Chin-Yew Lin. 2016. Knowledge base completion via coupled path ranking. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Berlin, Germany, 1308–1318.
- [45] Quan Wang, Zhendong Mao, Bin Wang, and Li Guo. 2017. Knowledge graph embedding: A survey of approaches and applications. *IEEE Transactions on Knowledge and Data Engineering* 29, 12 (2017), 2724–2743.
- [46] Song Wang, Xiao Huang, Chen Chen, Liang Wu, and Jundong Li. 2021. *REFORM: Error-Aware Few-Shot Knowledge Graph Completion*. Association for Computing Machinery, New York, NY, USA, 1979–1988. <https://doi.org/10.1145/3459637.3482470>
- [47] Yaqing Wang, Fenglong Ma, and Jing Gao. 2020. Efficient Knowledge Graph Validation via Cross-Graph Representation Learning. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (Virtual Event, Ireland) (CIKM '20)*. ACM, New York, USA, 1595–1604. <https://doi.org/10.1145/3340531.3411958>

- 1145/3340531.3411902
- [48] Gabriel Weaver, Barbara Strickland, and Gregory Crane. 2006. Quantifying the Accuracy of Relational Statements in Wikipedia: A Methodology. In *Proceedings of the 6th ACM/IEEE-CS Joint Conference on Digital Libraries* (Chapel Hill, NC, USA) (*JCDL '06*). ACM, New York, USA, 358. <https://doi.org/10.1145/1141753.1141853>
- [49] Dominik Wienand and Heiko Paulheim. 2014. Detecting incorrect numerical data in dbpedia. In *European Semantic Web Conference*. Springer, Cham, 504–518.
- [50] Haiqin Yang, Irwin King, and Michael R Lyu. 2010. Multi-task learning for one-class classification. In *IEEE IJCNN*. IEEE, Barcelona, Spain, 1–8.
- [51] Yu Zhao, Huali Feng, and Patrick Gallinari. 2019. Embedding learning with triple trustiness on noisy knowledge graph. *Entropy* 21, 11 (2019), 1083.
- [52] Li Zheng, Zhenpeng Li, Jian Li, Zhao Li, and Jun Gao. 2019. AddGraph: Anomaly Detection in Dynamic Graph Using Attention-based Temporal GCN. In *IJCAI*. IJCAI, USA, 4419–4425.